CESDE MEDELLÍN · ESCUELA DE TECNOLOGÍA · MOMENTO 2

🐄 PROGANADO S.A.S.

Sistema Modular de Información y Gestión Ganadera Bovina para el Hato Antioqueño

ASIGNATURA:	DOCENTE EVALUADOR:	ESPACIO / AULA:	FECHA DE SUSTENTACIÓN:
Introducción a la Programación (FTCOCU-129)	Ing. Jonathan Felipe Piedrahita Nieto	Aula 406 (Sede Principal CESDE Medellín)	Sábado 07 de Noviembre de 2026

📋 Ficha Técnica del Equipo de Desarrollo (Roles de Ingeniería)

INTEGRANTE	ROL EN EL PROYECTO	RESPONSABILIDAD CENTRAL Y APORTE TÉCNICO
Jeiser Gutiérrez	LÍDER TÉCNICO & FRONTEND	Arquitectura SPA con React 19 + Vite + Tailwind CSS, diseño de componentes modulares, ciclo de vida y optimización.
Sebastián Correa	REQUERIMIENTOS & BACKEND	Diseño de la estructura de datos relacional, modelado de entidades zootécnicas y especificación de endpoints API REST.
Emilio Villanueva	DISEÑO UI/UX & FLUJOS	Construcción de wireframes, experiencia de usuario responsive para dispositivos móviles de campo y paleta visual.
Dr. Héctor Pinto	EXPERTO TEMÁTICO ZOOTECNIA	Validación zootécnica de parámetros lecheros, períodos de retiro ICA, control de mastitis y normatividad ganadera colombiana.

🌐 Enlaces Oficiales y Entorno de Producción

El producto digital del Momento 2 ha sido compilado, verificado y publicado formalmente para acceso público y revisión académica inmediata:

- **Despliegue Web en Vivo (Producción CDN):** https://jeiserlabs.github.io/proganado-producto-momento-2/
- **Repositorio Oficial de Código Fuente (GitHub):** `jeiserlabs/proganado-producto-momento-2` (Rama: `main`).
- **Peso Total de la Hoja de Estilos de Producción: 24.6 kB** (`dist/assets/index.css` con Tree-Shaking verificado).
- **Peso del Bundle JavaScript: 273 kB** (`dist/assets/index.js` con React 19 y Lucide Icons).

🎯 Propósito y Resumen Ejecutivo del Documento

Este documento constituye la **Guía Maestra y Definitiva de Sustentación** para el Momento 2 de Introducción a la Programación. Su propósito es erradicar explicaciones abstractas o memorísticas y sustituirlas por **principios formales de ingeniería de software explicados a través de analogías universales de la vida real**.

Cualquier miembro del equipo, directivo académico o docente que lea esta guía comprenderá con exactitud matemática qué es el DOM, cómo funciona el Virtual DOM de React, por qué separamos la aplicación en componentes modulares, cómo viajarán los datos en el Momento 3 a través de una API REST protegida con esquemas de validación Zod, y cómo la arquitectura en la nube de tres capas garantiza que el sistema funcione fluidamente incluso en zonas ganaderas con baja conectividad móvil.

1. El Gran Salto de Calidad: De Bootstrap Monolítico a React 19 + Tailwind Atómico

👔 La Gran Metáfora: El Traje Prefabricado de Tienda vs La Caja de Piezas LEGO

Para entender la decisión arquitectónica que tomó ProGanado, imaginemos dos formas completamente diferentes de construir:

- **Bootstrap 5 es como comprar un traje prefabricado en una tienda por departamentos:** Viene con un corte rígido predeterminado. Si todos los alumnos del salón compran el mismo traje, todos se ven exactamente iguales. Si a ti el pantalón te queda un poco largo o quieres cambiar el color de un solo botón, tienes que pelear con tijeras, alfileres y costuras forzadas. En el mundo del código, esto equivale a llenar los archivos de sobreescrituras dolorosas y reglas sucias con `!important` para poder cambiar la apariencia de un botón.
- **Tailwind CSS es como recibir una caja infinita de piezas LEGO de precisión:** No te entrega un castillo terminado; te entrega las piezas atómicas fundamentales: un bloque verde esmeralda (`bg-[#198754]`), una curva suave para las esquinas (`rounded-xl`), una distancia exacta de relleno (`p-4`) y una orden de alineación flexible (`flex items-center`). Con estas piezas elementales construyes exactamente el diseño que el hato ganadero necesita, sin pelear contra plantillas ajenas.

⚡ La Métrica Real de Ingeniería: El Concepto de "Tree-Shaking" (Sacudir el Árbol)

En el software profesional, el rendimiento no es una opinión; es una medición matemática en bytes:

- **El Problema del Desperdicio con Bootstrap:** Cuando una página web incluye Bootstrap, el navegador se ve obligado a descargar un archivo gigantesco que contiene estilos para carruseles, acordeones, barras de progreso y tablas que el proyecto jamás utilizará. Es como contratar un camión de carga de 10 toneladas para transportar una sola bolsa de café.
- **La Solución Quirúrgica de Tailwind + Vite:** Tailwind funciona mediante un analizador estático que examina cada línea de nuestros archivos `.jsx` . Durante el proceso de compilación (`npm run build`), el compilador aplica una técnica llamada **Tree-Shaking** (sacudir el árbol): todas las clases de CSS que no hayamos escrito expresamente en el código quedan podadas y eliminadas del archivo final.
- **El Resultado Medido en ProGanado:** La hoja de estilos final de nuestra plataforma pesa tan solo **24.6 kB** (`dist/assets/index.css`). Un archivo de 24 kB descarga en **menos de 80 milisegundos** en una conexión móvil 3G o EDGE en fincas de Santa Rosa de Osos o San Pedro de los Milagros.

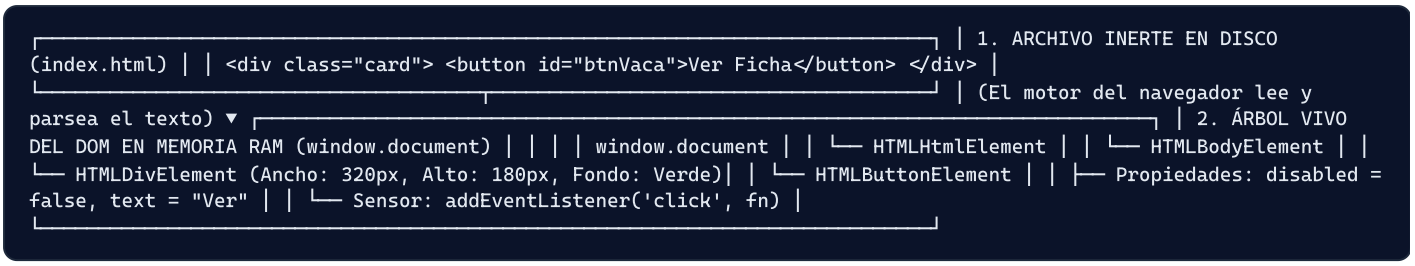
CRITERIO DE EVALUACIÓN	ENFOQUE TRADICIONAL (BOOTSTRAP 5)	ENFOQUE MODERNO PROGANADO (TAILWIND CSS 3.4)
Filosofía de Diseño	Componentes monolíticos rígidos prefabricados (`.btn` , `.card`).	Utilidades atómicas componibles (`flex` , `p-4` , `rounded-xl`).
Peso de Estilos en Red	Pesado (~280 kB a 350 kB de CSS con reglas muertas).	Ultraligero: **24.6 kB** gracias a la purga de estilos en Vite.
Identidad Visual	Genérica. La web se parece a millones de sitios existentes.	100% personalizada con identidad agropecuaria verde esmeralda.
Mantenimiento	Frágil. Cambiar un estilo rompe otras vistas por cascada.	Seguro. Las clases viven encapsuladas en su propio componente.

2. El Navegador y el DOM: Del Plano Inerte en Papel a la Casa Viva en Memoria

🏠 La Gran Metáfora: El Plano en Papel vs La Casa Física con Tuberías y Luz

Muchos estudiantes confunden el archivo HTML con la página web. La diferencia es abismal:

- **El archivo `index.html` es solo un plano arquitectónico dibujado en papel:** Está guardado en el disco duro. Son letras y símbolos inertes. Si tú tocas con el dedo el dibujo de una puerta en un plano de papel, la puerta no se abre, ni suena un timbre, ni se enciende una luz. Es un documento muerto de dos dimensiones.
- **El motor del navegador (V8 en Chrome o WebKit en Safari) es el equipo de construcción:** Cuando tú abres el archivo, el navegador lee las instrucciones escritas en el plano y **construye la casa física dentro de la memoria RAM del computador**.
- **El DOM (Document Object Model) es la casa física ya construida:** En la memoria RAM, cada etiqueta `<div>`, `<button>` o `<h1>` se transforma en un **objeto vivo de software**. Este objeto tiene dimensiones reales en píxeles, sabe en qué coordenada de la pantalla está parado, tiene cables eléctricos conectados (detectores de eventos como `onClick`) y reacciona de inmediato cuando el usuario interactúa con él.



👂 ¿Por qué decimos que los nodos del DOM "tienen oídos y sensores"?

En JavaScript tradicional, el navegador dota a cada elemento de la pantalla de la capacidad de escuchar lo que hace el usuario mediante un sistema de suscripción a eventos:

- **Eventos de Ratón / Táctiles (`onClick`, `onMouseEnter`):** El botón detecta la presión física del dedo sobre la pantalla táctil de un teléfono o el clic del mouse y despierta la función asociada.
- **Eventos de Teclado (`onKeyDown`, `onChange`):** En el buscador de ProGanado, cada vez que el usuario teclea una letra (por ejemplo "M-a-n-u-e-l-a"), el nodo emite una señal eléctrica de software para filtrar el catálogo en tiempo real.
- **Eventos de Formulario (`onSubmit`):** Capturan el envío del modal de registro de bovinos y previenen la recarga obligatoria de la página mediante `e.preventDefault()`.

3. El Virtual DOM de React: El Gemelo Digital del Arquitecto

⚠️ El Grave Problema del DOM Tradicional: Lentitud y Calentamiento de Celulares

Cuando un programador novato utiliza JavaScript tradicional para cambiar el contenido de la pantalla (`document.getElementById('vacas').innerHTML = nuevoHTML`), el navegador se ve obligado a destruir los elementos viejos y reconstruir la vista desde cero. Este proceso desata dos operaciones extremadamente pesadas para el procesador:

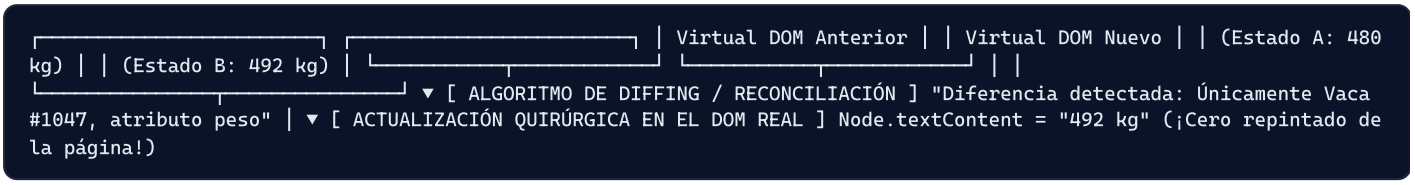
- **Reflow (Recálculo de Geometría):** El navegador tiene que recalcular las coordenadas, anchos, márgenes y sombras de cada elemento de la pantalla.
- **Repaint (Repintado de Píxeles):** La tarjeta gráfica debe volver a pintar millones de píxeles en el monitor o pantalla del celular.

Si una finca tiene un listado de 500 animales y el usuario teclea una letra en el buscador, el navegador intentaría hacer 500 destrucciones y reconstrucciones por segundo. La pantalla se congela, los botones no responden y la batería del celular se drena rápidamente.

📐 La Gran Metáfora: El Gemelo Digital en la Tableta Gráfica del Arquitecto

Para solucionar este problema de raíz, los ingenieros de React inventaron el concepto de **Virtual DOM**:

- **El error de picar la pared real con una maza:** Imagina un arquitecto remodelando una casa. Si cada vez que tiene una duda sobre dónde poner una ventana manda a los obreros a demoler la pared física de ladrillo con mazos, la casa se debilita, se llena de polvo y los costos se disparan. Eso es lo que hace JavaScript viejo manipulando el DOM real directamente.
- **El Gemelo Digital en la Tableta (Virtual DOM):** El arquitecto moderno abre su tableta gráfica con un modelo 3D liviano de la casa. En la tableta puede mover la ventana 50 veces en 2 segundos, probar colores y cambiar la distribución sin gastar un solo ladrillo ni hacer ruido. Ese modelo digital liviano en la memoria RAM de JavaScript es el **Virtual DOM**.
- **El Algoritmo de Reconciliación (Diffing Algorithm):** Cuando el usuario interactúa con ProGanado, React genera un nuevo gemelo digital y lo compara matemáticamente contra el gemelo digital del segundo anterior. Detecta con precisión quirúrgica: *"De las 50 vacas mostradas, 49 siguen exactamente iguales. Solo la vaca #1047 cambió su pesaje de 480 kg a 492 kg"*.
- **La Actualización Quirúrgica:** React acude al DOM real del navegador y le aplica una inyección directa únicamente al texto de esos 12 caracteres. El resto de la casa permanece intacta, garantizando una fluidez de 60 fotogramas por segundo ininterrumpidos.



4. Modularidad y Componentes: La Cadena de Ensamblaje Automotriz

🚗 La Gran Metáfora: La Ensambladora de Vehículos y el Principio SRP

En el desarrollo de software antiguo, se solía escribir todo el código de una página en un único archivo monstruoso de 4.000 líneas. Si se dañaba una coma en la línea 3.500, toda la página dejaba de funcionar por completo.

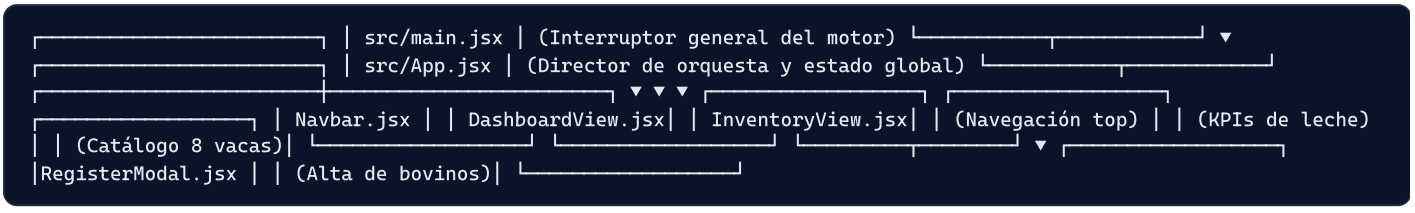
React resolvió esto aplicando la misma filosofía de una **planta de ensamblaje de vehículos modernos**:

- **Piezas Independientes:** Un automóvil Toyota no se funde como un solo bloque macizo de metal. Se fabrica el motor por separado, la caja de velocidades por separado, las puertas por separado y el tablero de instrumentos por separado.
- **Principio de Responsabilidad Única (Single Responsibility Principle - SRP):** Cada pieza tiene una única misión en la vida y la cumple de forma excelente. Si el velocímetro del tablero sufre una falla, el mecánico no tiene que desmontar el tanque de gasolina ni rehacer los frenos; simplemente retira el módulo del tablero y lo reemplaza.

🧩 Mapeo de la Cadena de Montaje en ProGanado Web

Nuestra aplicación ganadera fue diseñada bajo una arquitectura modular limpia de componentes independientes ubicados en la carpeta `src/components/`:

COMPONENTE JSX	ANALOGÍA AUTOMOTRIZ	RESPONSABILIDAD ESPECÍFICA EN PROGANADO
`Navbar.jsx`	**El Tablero de Mandos**	Contiene la identidad institucional, la barra de estado de navegación y los botones de conmutación entre vistas.
`DashboardView.jsx`	**El Odómetro y Tacómetro**	Presenta las métricas zootécnicas clave: litros diarios producidos, acopio entregado a Colanta y alertas tempranas.
`InventoryView.jsx`	**El Maletero e Inventario**	Despliega el catálogo de animales con tarjetas individuales, barra de búsqueda reactiva y filtros por raza y potrero.
`HealthModule.jsx`	**El Sistema de Frenos ABS**	Módulo de seguridad zoosanitaria: gestiona los períodos de retiro oficial del ICA y bloquea el ordeño de vacas medicadas.
`RegisterModal.jsx`	**La Ventanilla de Registro**	Formulario modal flotante para dar de alta nuevos ejemplares bovinos con validación inmediata de campos obligatorios.



5. Anatomía de la Carpeta del Proyecto: La Línea de Empaque y Despliegue

Para defender el proyecto ante el docente evaluador, cada integrante debe conocer con precisión milimétrica la función exacta de cada archivo que conforma el ecosistema de ProGanado Web:

ARCHIVO O CARPETA	ANALOGÍA TÉCNICA	¿QUÉ HACE POR DENTRO Y POR QUÉ ES INDISPENSABLE?
`index.html`	**La Puerta de Entrada**	Es el único archivo HTML de todo el proyecto. El navegador lo busca por defecto. Contiene una sola línea crucial: `<div id="root"></div>` y una etiqueta de script modular que llama a `main.jsx`. ("Root" significa raíz del árbol visual).
`src/main.jsx`	**El Interruptor de Encendido**	Es un archivo conciso de apenas 10 líneas. Inicializa la librería React mediante `ReactDOM.createRoot`, inyecta los estilos globales de `index.css` y monta el componente maestro `<App />` en el contenedor `#root`.
`src/App.jsx`	**El Director de Orquesta**	Controla la lógica de la pantalla: mantiene la variable de estado `activeTab` ("dashboard", "inventory", "health"), orquesta la apertura del modal y sincroniza los cambios de datos con el almacenamiento local del navegador (`localStorage`).
`src/data/bovinosData.js`	**La Despensa Provisional (Mock)**	Un archivo de datos estructurados en JavaScript con 8 registros bovinos reales (arete zootécnico, raza, fecha de parto, litros/día, lote y estado ICA). Permite que la aplicación sea 100% funcional y demostrable en el aula sin requerir internet ni conexión a servidor.
`package.json`	**La Cédula y Receta Técnica**	Es el manifiesto del proyecto. Especifica el nombre, la versión, las dependencias de producción (React 19, Lucide React) y los scripts ejecutables que usamos en la terminal (`dev`, `build`, `preview`).
`package-lock.json`	**El Candado Criptográfico**	Registra el árbol exacto de dependencias con firmas criptográficas SHA-512. Garantiza que si otro estudiante o el docente clona el proyecto, se descarguen exactamente las mismas versiones de librerías sin romper el código.
`vite.config.js`	**El Plano de la Refinería**	Le indica al empaquetador Vite qué plugins debe activar (en nuestro caso, el plugin oficial de React) y cómo optimizar los bundles de código final.
`tailwind.config.js`	**La Paleta de Identidad**	Define los colores oficiales de ProGanado (verde esmeralda `#198754`, verde oscuro de hato `#062416`), las fuentes tipográficas y los puntos de quiebre responsive para pantallas móviles.
`postcss.config.js`	**El Traductor de Estilos**	Conecta el compilador de Tailwind con el motor de CSS del navegador y aplica `autoprefixer` para que los estilos funcionen igual en Chrome, Firefox, Safari y Edge.
`dist/`	**La Fábrica de Producción**	Carpeta autogenerada tras ejecutar `npm run build`. Contiene el HTML, CSS y JavaScript minificados y empaquetados, listos para desplegar en GitHub Pages o cualquier servidor web mundial.

6. npm vs pnpm y la Orquestación de Vite: La Refinería del Navegador

¿Por qué usamos npm y no pnpm en este proyecto?

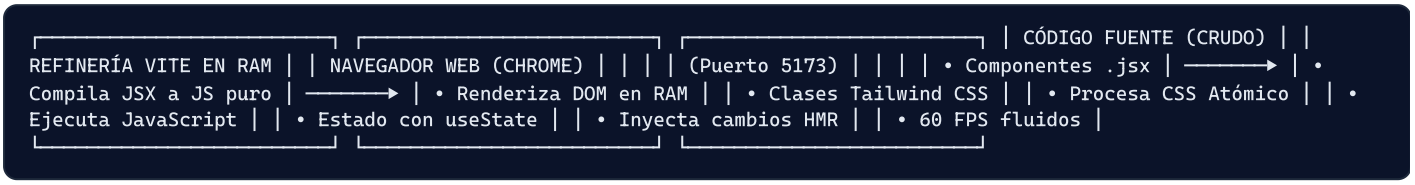
Ambas son herramientas excelentes de gestión de paquetes en el ecosistema Node.js, pero obedecen a arquitecturas de almacenamiento diferentes:

- **La Metáfora de las Bibliotecas:**
  - **npm es como comprar libros físicos y poner una copia en cada aula de clase:** Cada proyecto tiene su propia carpeta física `node_modules`. Es un método 100% aislado, compatible con cualquier versión de Windows y garantizado en cualquier computador institucional del CESDE sin requerir privilegios de superusuario ni configuraciones especiales.
  - **pnpm es como una gran biblioteca central indexada con casilleros inteligentes:** Descarga cada versión de una librería una sola vez en el disco duro y en los proyectos coloca accesos directos (*hard links* y *symlinks*). Es sumamente rápido y ahorra espacio, pero en laboratorios universitarios con políticas de seguridad estrictas en Windows, los permisos de enlaces simbólicos suelen estar bloqueados, provocando errores en cascada.
- **La Decisión de Ingeniería:** Usamos `npm` porque priorizamos la **estabilidad, compatibilidad universal y cero fallos** durante la sustentación académica.

¿Por qué corremos "npm run dev" y no ejecutamos directamente main.jsx o App.jsx?

Esta es una de las preguntas conceptuales más importantes de toda la carrera técnica:

- **Los navegadores web son animales de costumbres fijas:** Un navegador como Google Chrome o Microsoft Edge **solo sabe interpretar tres lenguajes nativos:** HTML plano, CSS plano y JavaScript estándar (ECMAScript).
- **Los navegadores NO entienden JSX de forma nativa:** Si tú intentas abrir un archivo con etiquetas como `<Navbar activeTab={tab} />` directamente en el navegador con doble clic o con la extensión "Live Server", el motor de JavaScript lanzará un error de sintaxis inmediato (`Uncaught SyntaxError: Unexpected token '<'`). El navegador no sabe qué significa esa sintaxis de etiquetas combinada con lógica de programación.
- **Vite actúa como la Refinería de Combustible en tiempo real:** Un motor diésel no funciona con petróleo crudo recién extraído del pozo petrolero; necesita una refinería intermedia que transforme el crudo en combustible puro. **Vite es esa refinería:**
  - Cuando ejecutamos `npm run dev`, Vite levanta un servidor de desarrollo ultraoptimizado en la memoria RAM en el puerto 5173 (`http://localhost:5173/`).
  - Intercepta cada petición del navegador, toma el código JSX crudo y las clases de Tailwind, las compila y transforma instantáneamente en JavaScript y CSS estándar, y se las entrega al navegador ya digeridas.
  - Gracias a la tecnología **HMR (Hot Module Replacement)**, si modificamos el código en VS Code, Vite actualiza únicamente ese componente en la pantalla en menos de **50 milisegundos** sin necesidad de refrescar la página.



7. El Viaje de los Datos: La Gran Metáfora del Restaurante (APIs, Endpoints y Zod)

🚫 Por qué un Navegador Web JAMÁS debe conectarse directo a una Base de Datos

Muchos estudiantes novatos preguntan: "*¿Por qué no ponemos la contraseña de la base de datos SQLite o PostgreSQL directamente en el archivo JavaScript del frontend?*". Hacer eso en producción es un **suicidio de ciberseguridad**. Todo el código que corre en el navegador es público: cualquier usuario puede presionar la tecla `F12`, abrir la pestaña "Fuentes", copiar la contraseña de la base de datos y ejecutar una instrucción destructiva como `DROP TABLE bovinos;`.

Por esta razón, la industria del software utiliza una **arquitectura cliente-servidor desacoplada** basada en el modelo de un restaurante.

🍽️ La Gran Metáfora del Restaurante de Alta Cocina

En ProGanado, la interacción de datos que conectaremos en el Momento 3 funciona con los siguientes roles:

- **1. El Cliente en la Mesa (El Frontend con React):** Es el mayordomo o ganadero sentado en el comedor con su teléfono móvil. No tiene acceso a la cocina ni sabe cómo están encendidos los fogones; solo interactúa con la interfaz visual.
- **2. El Menú del Restaurante (Los Endpoints de la API):** El cliente solo puede pedir lo que está formalmente escrito en el menú. Los endpoints son las rutas oficiales expuestas por el servidor:
  - `GET /api/bovinos`: Consultar la lista completa del hato.
  - `POST /api/pesajes`: Registrar los litros de leche ordeñados en el día.
  - `PUT /api/sanidad/1047`: Actualizar el estado de cuarentena por mastitis.
- **3. El Pasillo y el Mozo de Transporte (La función `fetch()`):** Es el conducto seguro de comunicación por el cual viaja la comanda a través de la red utilizando el protocolo HTTP/HTTPS.
- **4. El Mesero (La API REST en el Backend):** Es el intermediario autorizado. Recibe la petición del cliente y camina con la comanda en la mano hacia la cocina. El cliente jamás entra a la cocina; el mesero es el único que cruza la puerta.
- **5. El Inspector de Sanidad y Bromatología en la puerta de la cocina (Zod):** Antes de que la comanda entre a los sartenes, el inspector la examina con lupa. Si un atacante malicioso escribió en la comanda caracteres extraños para engañar al cocinero (`' OR 1=1 --`), **Zod detecta que la entrada no cumple el esquema formal** y la rechaza inmediatamente con un error `400 Bad Request`, impidiendo cualquier intento de *SQL Injection*.
- **6. La Cocina y la Despensa (La Base de Datos SQLite / PostgreSQL):** El chef recibe la comanda limpia y validada, abre la despensa, consulta los registros y cocina el plato solicitado.
- **7. El Plato Servido en Bandeja Limpia (El archivo JSON):** La comida preparada no se entrega en costales sucios; se sirve en una bandeja elegante y estandarizada: un archivo de texto ligero estructurado en formato **JSON**, que el mesero lleva de vuelta a la mesa del cliente para que React lo dibuje en la pantalla.

```
[ CLIENTE EN LA MESA ] ⟶ El mayordomo usando React en su celular o tablet. | ▼ (Selecciona una opción del MENÚ: GET /api/bovinos) [ EL PASILLO ] ⟶ La función fetch() enviando la petición por HTTPS. | ▼ [ EL MESERO ] ⟶ LA API REST (Intermediario seguro del servidor). | ▼ [ INSPECTOR SANITARIO ] ⟶ ZOD (Valida tipos de datos y bloquea SQL Injection). | ▼ [ LA COCINA / DESPENSA ] ⟶ BASE DE DATOS SQL (Ejecuta SELECT * FROM bovinos;). | ▼ [ BANDEJA SERVIDA ] ⟶ EL JSON (Texto plano ligero de 10 kB que viaja al celular).
```

8. Arquitectura Cloud en Tres Capas: La Bodega Mayorista vs El Mostrador

📉 El Escenario Real: ¿Por qué jamás guardar 50.000 fotos en la carpeta del Frontend?

Imaginemos una empresa ganadera a gran escala con 50.000 reses registradas. Si cometiéramos el error de principiante de almacenar los archivos de foto de alta resolución dentro de la carpeta del frontend, ocurriría una **catástrofe técnica**:

- Cada foto en buena resolución pesa aproximadamente 1 Megabyte. Multiplicado por 50.000 vacas, el frontend pesaría **50 Gigabytes**.
- Cuando un mayordomo en una vereda apartada de Antioquia abra la página web en su teléfono con señal celular débil (3G/HSPA), el navegador intentaría descargar los 50 GB de golpe.
- **Consecuencias:** El navegador colapsaría por falta de memoria RAM, se consumiría todo el plan de datos mensual del trabajador en dos minutos y la batería del dispositivo se sobrecalentaría.

☁️ La Solución en Tres Capas Desacopladas de ProGanado

Para garantizar escalabilidad industrial, ProGanado divide su arquitectura en tres capas independientes y especializadas:

Capa 1: Mostrador (CDN)

`FRONTEND LIVIANO`

El código de React y Tailwind se hospeda en redes globales de distribución (CDN). Pesa solo 300 kB y carga en 0.4 segundos. Implementa **paginación inteligente**: solo le pide al servidor los 10 o 20 animales que caben en la pantalla del usuario.

Capa 2: Cómputo (Lambda)

`SERVERLESS ON-DEMAND`

En vez de tener un servidor dedicado encendido las 24 horas gastando electricidad mientras todos duermen, usamos funciones bajo demanda (AWS Lambda). Solo despiertan durante 200 milisegundos cuando se registra un pesaje o alerta ICA y vuelven a dormir ($0 costo en reposo).

Capa 3: Bodega (S3 + SQL)

`ALMACÉN DE OBJETOS`

Las fotos pesadas se guardan en un almacén de objetos especializado en la nube (AWS S3). En la base de datos SQL no se guarda la foto; **solo se guarda una cadena de texto corta con la dirección web (URL)** de 50 caracteres: `https://s3.../vaca_1047.webp`.

VARIABLE OPERATIVA	ARQUITECTURA MONOLÍTICA (TODO EN FRONTEND)	ARQUITECTURA CLOUD 3 CAPAS (PROGANADO)
Tamaño de Descarga Inicial	Catastrófico (> 50 Gigabytes con 50.000 vacas).	Óptimo: **Menos de 300 Kilobytes**.
Consumo de Memoria RAM	Satura y congela dispositivos móviles de campo.	Estable y constante (~45 MB de RAM en Chrome móvil).
Costo de Servidores en Reposo	Costoso (servidor encendido 24/7 cobrando mensualidad).	**$0 COP en reposo** gracias a la arquitectura Serverless.
Velocidad en Redes Rurales 3G	Inviable (tiempo de carga superior a 40 minutos).	Instantánea (carga en menos de 0.8 segundos).

9. Manual Operativo de Comandos: El Ciclo de Vida de Vite en el Taller

Para la sesión práctica de hoy y la sustentación en los computadores del CESDE, el equipo debe dominar los tres comandos oficiales configurados en nuestro archivo `package.json`:

🛠️ Los Tres Comandos Maestros de Vite

COMANDO DE TERMINAL	NOMBRE DEL MODO	¿QUÉ HACE INTERNAMENTE Y CUÁNDO SE UTILIZA?
`npm run dev`	**Modo Taller Interactivo (Desarrollo)**	Levanta el servidor local de desarrollo en el puerto 5173 (`http://localhost:5173/`). Compila el código JSX en tiempo real en la memoria RAM y activa la sincronización instantánea HMR. *(¡Este es el comando obligatorio para programar y sustentar en clase!)*.
`npm run build`	**Modo Fábrica de Producción**	Toma todos los archivos fuente, ejecuta el analizador de Rollup y Tailwind, aplica Tree-Shaking para eliminar estilos no usados, minifica el JavaScript y genera la carpeta física `dist/` lista para publicar en la nube.
`npm run preview`	**Modo Ensayo General Previo**	Levanta un servidor local en el puerto 4173 (`http://localhost:4173/`) que sirve directamente los archivos compilados de la carpeta `dist/` . Permite auditar exactamente cómo se comportará la página web en producción antes de subirla a internet.

🚑 Guía de Primeros Auxilios: Solución a Errores Comunes en el Aula 406

Si durante la sustentación en la computadora del laboratorio ocurre algún incidente técnico, el equipo aplicará de inmediato estas soluciones probadas:

- **Incidente 1: "Port 5173 is in use, trying another one...":**
  Causa: Otro compañero o proceso dejó abierto un servidor en ese puerto.
  Solución: Vite automáticamente abrirá el puerto 5174 o 5175. Simplemente mira la terminal y haz clic en el enlace azul que aparece en pantalla (`http://localhost:5174/`).
- **Incidente 2: "sh: vite: command not found" o carpetas vacías:**
  Causa: La computadora del laboratorio no tiene descargadas las librerías locales.
  Solución: Escribe en la terminal `npm install` , presiona Enter, espera 20 segundos a que se reconstruya la carpeta `node_modules` y vuelve a ejecutar `npm run dev` .
- **Incidente 3: Pantalla en blanco con error en consola F12:**
  Causa: Alguien intentó abrir la página con "Live Server" o haciendo doble clic directo sobre `index.html` .
  Solución: Recuerda la regla de oro: **JSX requiere la refinería de Vite**. La página solo abre a través de la dirección `http://localhost:5173/` levantada por la terminal.

10. Banco de 12 Preguntas de Fuego del Jurado y Respuestas Maestras (Para Sacar 5.0)

Este cuestionario contiene las preguntas técnicas más rigurosas que el docente Ing. Jonathan Felipe Piedrahita Nieto o los compañeros del curso pueden formular durante la sustentación, con su respectiva respuesta de ingeniería:

1. ¿Por qué utilizaron la extensión de archivo .jsx en lugar de .js convencional?
Respuesta: Porque JSX es una extensión sintáctica que permite escribir etiquetas de interfaz declarativa directamente dentro de la lógica de JavaScript. La extensión `.jsx` le indica explícitamente a Vite y a los compiladores modernos que deben activar los procesadores de transformación de interfaz de React antes de empaquetar el módulo.

2. ¿Qué ventaja ofrece un componente funcional frente a una plantilla HTML tradicional?
Respuesta: Un componente funcional en React es una función pura de JavaScript reutilizable que acepta parámetros de entrada (`props`) y retorna un fragmento de interfaz reactiva. Permite encapsular lógica, estado y diseño en un solo bloque autocontenido, evitando la duplicación de código.

3. ¿Qué sucedería si abrimos index.html directamente con doble clic en el explorador de archivos?
Respuesta: La página aparecería en blanco y el navegador arrojaría errores de seguridad CORS y de sintaxis. Los navegadores bloquean la importación de módulos ES (`<script type="module">`) bajo el protocolo local `file:///` y no pueden traducir el JSX sin el servidor local de desarrollo de Vite.

4. ¿Cómo gestiona ProGanado la persistencia de datos en esta maqueta sin tener un backend activo?
Respuesta: Implementamos una capa de persistencia dual: cargamos un catálogo inicial zootécnico desde `src/data/bovinosData.js` y sincronizamos los nuevos registros agregados por el usuario directamente en la memoria `localStorage` del navegador mediante los hooks de React. Si el usuario recarga la página, los datos registrados permanecen intactos.

5. ¿Cuál es la diferencia fundamental entre el "Estado" (state) y las "Propiedades" (props) en React?
Respuesta: Las `props` son datos inmutables que un componente padre le pasa a un componente hijo (como el arete o la raza de una vaca). El `state` (manejado con `useState`) es la memoria interna y mutable del componente; cuando el estado cambia, React programa automáticamente una actualización quirúrgica en la pantalla.

6. ¿Por qué Tailwind CSS es más eficiente para una aplicación móvil ganadera que Bootstrap?
Respuesta: Porque Tailwind compila únicamente las clases que efectivamente utilizamos mediante Tree-Shaking, produciendo una hoja de estilos de solo **24.6 kB** frente a los más de 300 kB de Bootstrap. Esto garantiza una velocidad de carga instantánea en teléfonos celulares operando en zonas rurales con conectividad limitada.

7. ¿Cómo protege la librería Zod la integridad de la base de datos contra ataques de inyección SQL?
Respuesta: Zod actúa como un validador de esquemas de tipos estricto en tiempo de ejecución. Antes de que una comanda enviada desde el frontend llegue al motor SQL, Zod comprueba que los textos, números y fechas coincidan con el contrato esperado. Si un atacante intenta inyectar sentencias SQL o caracteres no permitidos, la petición es abortada de inmediato en la puerta de entrada.

8. ¿Qué significa formalmente que la aplicación web sea una SPA (Single Page Application)?
Respuesta: Significa que el navegador web solo descarga un único archivo HTML (`index.html`) durante toda la sesión. Cuando el usuario navega entre "Panel de Control", "Inventario" o "Sanidad", la página nunca se recarga en blanco; React intercambia dinámicamente los componentes en la memoria RAM, ofreciendo la experiencia fluida de una aplicación nativa de escritorio.

11. Glosario Técnico de Ingeniería de Software

• **DOM (Document Object Model):** Representación viva en memoria RAM de un documento HTML en forma de árbol de objetos con propiedades y detectores de eventos sensoriales.

• **Virtual DOM:** Réplica liviana del DOM en memoria de JavaScript que permite calcular diferencias y ejecutar actualizaciones quirúrgicas sin repintar la pantalla.

• **Tree-Shaking:** Proceso de optimización de compiladores que analiza el código fuente y elimina todas las funciones y estilos CSS que no se utilizan en el bundle final.

• **Hot Module Replacement (HMR):** Mecanismo de Vite que inyecta módulos actualizados en el navegador en tiempo real sin recargar la página ni perder el estado actual.

• **API REST:** Interfaz de programación basada en el protocolo HTTP que define un conjunto de reglas y endpoints para el intercambio estructurado de datos en formato JSON.

• **Zod Schema:** Librería de declaración y validación de tipos en TypeScript/JavaScript para asegurar que los datos de entrada cumplan estrictamente con las reglas de negocio.

12. Acta de Entrega y Evaluación Académica (Momento 2)

Por medio de la presente, se certifica la entrega formal del **Producto de Software del Momento 2** de la asignatura **Introducción a la Programación (FTCOCU-129)** correspondiente a la maqueta funcional interactiva de la plataforma **ProGanado S.A.S.**, cumpliendo con todos los criterios de diseño modular, separación de capas y responsabilidades técnicas.

ESTUDIANTE / INTEGRANTE	DOCUMENTO IDENTIDAD	FIRMA DE CONFORMIDAD
Jeiser Gutiérrez (Líder Técnico)	C.C. 1.019.156.838	_____
Sebastián Correa (Requerimientos)	Estudiante CESDE	_____
Emilio Villanueva (Diseño UI/UX)	Estudiante CESDE	_____
Dr. Héctor Pinto (Zootecnia)	Médico Veterinario / CESDE	_____

DICTAMEN Y EVALUACIÓN DEL DOCENTE:

Ing. Jonathan Felipe Piedrahita Nieto · Escuela de Tecnología CESDE

CALIFICACIÓN DEFINITIVA: [/ 5.0]

Observaciones del Evaluador: _____

Firma del Docente: _____